Universitatea Politehnica București (UPB) Facultatea de Automatică și Calculatoare

Algoritmi Paraleli și Distribuiți
Test Curs (Varianta Exemplu 2)
6 Decembrie 2024, ora 12:00
Timp de lucru: 45 de minute

Numele: _____

Grupa: _____

Reguli de desfășurare a testului:

- Tipul testului este **closed book**. Pe durata acestuia nu se permit consultarea materialelor de studiu, utilizarea dispozitivelor electronice sau colaborarea cu alte persoane.
- Testul cuprinde două probleme, ambele obligatorii.
- Pentru fiecare problemă veți primi o notă de la 0 la 100 (în procente), iar punctajul final se va calcula ca produsul dintre cele două punctaje.

  Exemplu: Cu 75% pe prima problemă și 90% pe a doua nota finală va fi: 67.5% (din punctajul testului).
- Foaia cu subiecte se vor preda obligatoriu odată cu rezolvările.

Problema 1
Completați pseudocodul din Listing-ul 1 astfel încât funcția foo să poată fi apelată de maxim 3 fire de execuție simultan. Pentru sincronizare se vor folosi semafoare.

```
process Apelant[tid = 1 to N] {
    foo();
}
```

Listing 1: Schelet problema 1.

Soluția propusă trebuie să maximizeze concurența și să nu impună alte restricții în afară de cea din enunț.

Problema 2[1].
Considerăm existența a două tipuri de fire de execuție (**Hidrogen** și **Oxigen**) și ne propunem ca acestea să formeze molecule de apă (din 2 fire de execuție **Hidrogen** și un thread **Oxigen**). Aderarea unui thread la o moleculă de apă se va face prin apelarea funcției **bound()**.

Fiecare thread trebuie să aștepte la o "barieră" implementată de voi ca numărul necesar de tread-uri din fiecare tip să ajungă la aceasta[2] înainte de a apela funcția **bound()**. Mai mult, formarea unei noi grupări de 3 fire de execuție se va face abia după ce cele 3 din molecula curentă au executat funcția **bound()**.

În alte cuvinte:

- Trebuie să garantăm că toate cele 3 fire de execuție au ajuns la barieră înainte ca oricare să apeleze funcția **bound**;
- Dacă un thread de tip oxigen ajunge la barieră și nu există fire de execuție hidrogen prezente, va aștepta 2 fire de execuție hidrogen;
- Dacă un fir de execuție de tip hidrogen ajunge la barieră și nu sunt prezente alte fire de execuție, el trebuie să aștepte un hidrogen și un oxigen.

Scrieți pseudocodul pentru cele 2 tipuri de fire de execuție (considerând câte N fire de execuție din fiecare tip). Pentru sincronizare se vor folosi bariere și/sau semafoare.

---

[1] Atât problema cât și soluția sunt preluate din cartea "The Little Book of Semaphores" de Allen B. Downey, disponibilă online la `https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf`

[2] La barieră.

Posibile soluții

Problema 1

```
// Variabile partajate
Sem s(3); // semafor initializat cu 3

process Apelant[tid = 1 to N] {
    P(s);
    foo();
    V(s);
}
```

Listing 2: Soluție pentru problema 1.

Problema 2

```
// Variabile partajate
Sem mutex(1), sem_o(0), sem_h(0);
int oxigen = 0, hydrogen = 0;
Bariera bariera(3);
// Definitia pentru firele de executie Oxigen
proces Oxigen[tid = 1 to N] {
    mutex.wait();
    oxigen += 1;
    if (hidrogen >= 2) {
        sem_h.signal(2);
        hidrogen -= 2;
        sem_o.signal();
        oxigen -= 1;
    } else {
        mutex.signal();
    }
    sem_o.wait();
    bound();
    bariera.wait();
    mutex.signal();
}
// Definitia pentru firele de executie Hidrogen
proces Hidrogen[tid = 1 to M] {
    mutex.wait();
    hidrogen += 1;
    if (hidrogen >= 2 and oxigen >= 1) {
        sem_h.signal(2);
        hidrogen -= 2;
        sem_o.signal();
        oxigen -= 1;
    } else {
        mutex.signal();
    }
    sem_h.wait();
    bound();
    bariera.wait();
    // semaforul mutex este eliberat de oxigen-ul din
    grupa
}
```

Listing 3: Soluție pentru problema 2.

Observații:

- Mai multe explicații privind soluția de mai sus găsiți în The Little Book of Semaphores (`https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf`);
- La curs, am evitat utilizarea wait(n) și signal(n) cu n diferit de 1. În acest caz, este în regulă să folosim signal/release(2) deoarece nu am crea probleme dacă am înlocui apelul cu 2 apeluri signal/release();
- Dacă nu se specifică altfel în cerință, la test nu se va folosi .acquire(n) sau wait(n) cu n diferit de 1.